



CS325 Software Engineering

Fall 2026 — Welcome to the course where you'll learn to build reliable software and manage projects effectively. This course covers the full software lifecycle, from requirements to deployment, with hands-on projects and modern industry practices.

GIK INSTITUTE

UNDERGRADUATE · CS

Logistics

Your instructor is available during office hours and through multiple channels. Reach out early if you have questions — don't wait until issues pile up.

Office Location

Room S18, 2nd Floor
New Academic Block

Office Hours

Tuesdays & Thursdays
10:00 AM – 11:00 AM
or by appointment

Email

farah.saeed@giki.edu.pk

WhatsApp

Reach out for quick queries and course-related questions.



Always include your name and section when contacting the instructor.

Logistics

Time and Location

In schedule

MS Teams

MS Teams code b3pgf0g

Credit hours: 3

CS325 – Software Engineering (SE)

Welcome to SE!

This course is about:

- **Learning to Build software that works well**
- **Learning to Manage projects that succeed.**
- **Gaining Hands-on experience**

Modern tools, strategies, generative AI

- To make coding, testing, and project planning easier.

Turning ideas into real solutions

- Doing it together in an organized, impactful way.

Software Engineering (SE) introduces students to the principles and practices of **building reliable software** and **managing projects effectively**. The course blends technical foundations with modern management strategies, equipping students to handle the full lifecycle of data-driven solutions while developing teamwork, leadership, and reproducibility skills.

CS311 – Course Learning Objectives (CLO)

CLO 1

Apply fundamental software engineering principles and project management techniques to formulate a comprehensive project plan for a given software problem.

CLO 2

Analyze various software development lifecycle (SDLC) models (e.g., Waterfall, Agile, Spiral) to select the most appropriate model for a specific software project based on its requirements, constraints, and risks.

CLO 3

Evaluate the quality of a software system by employing a multi-level testing strategy (unit, integration, system) to critically assess the results for identifying defects.

CLO 4

Create a collaborative software development environment by utilizing version control systems (like Git) along with project management software to track progress, manage changes, and facilitate team communication throughout the project lifecycle.

CS325 – CLOs Assessment Mechanism

Assessment	CLO_1	CLO_2	CLO_3	CLO_4
Quizzes	30%	0%	10%	0%
Assignments	40%	40%	30%	0%
Project	0%	0%	0%	100%
Mid Term	30%	30%	0%	0%
Final Term	0%	30%	60%	0%

GRADING

How the final grade breaks down

Weighted across five components, with the two exams carrying most of the grade.



■ Quizzes **10%**

■ Assignments **5%**

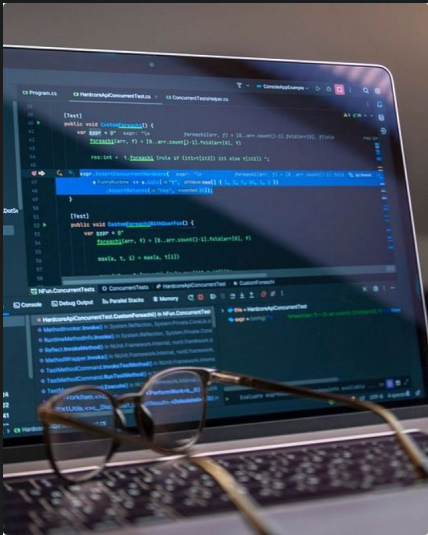
■ Projects **15%**

■ Mid Term **30%**

■ Final Term **40%**

Reference Materials

These books are your companions throughout the course. The primary textbook is Sommerville; the others are essential supplements that practicing engineers swear by.



Software Engineering — Ian Sommerville (11th Ed.)

The primary course textbook. Covers the full breadth of SE theory, process models, requirements, design, testing, and project management.



The Pragmatic Programmer

A timeless guide to thinking like a professional developer. Full of practical wisdom on craftsmanship, tooling, and career growth.



Clean Code — Robert C. Martin

Learn to write code that humans can read and maintain. Essential for understanding what separates good software from bad.

Course Content — 15-Week Roadmap

Weeks 1-2: Foundations

Intro to SE, ethics, SDLC, process models (Waterfall, Agile, Spiral), and case studies.

Weeks 3-4: Agile & DevOps

Scrum, Kanban, XP, Jira hands-on, Git, CI/CD pipelines.

Weeks 5-6: SDLC

Elicitation techniques, SRS, user stories, context and behavioral models.

Weeks 7-8: Design & Implementation

Architectural patterns, UML, design patterns, open source development.

Week 9: MVP Presentations

First major milestone — present your project's minimum viable product.

Weeks 10-11: Testing & Evolution

TDD, release testing, legacy systems, AI-assisted development.

Weeks 12-14: Project Management & SQA

WBS, Gantt charts, CPM, PERT, EVM, KPIs, quality assurance.

Week 15: Finals

Group project presentations, course review, and final exam.

Course Policies & Assessment

Assignments

Due on the date and time specified in the PDF. A few extra hours are allowed, but late submissions may not be accepted.

Quizzes

5 quizzes plus 2 retake opportunities. Academic dishonesty results in cancellation of the assessment.

Attendance

University policy requires **100% attendance**. Verify your attendance when marked. Updates require the batch advisor's signature.

Exams

University academic honesty policies apply. Violations carry serious consequences.



Always follow assessment instructions carefully — details matter.

Group Project

Your major deliverable is a **unique software project utilizing AI features**. The goal is to build something meaningful without relying on costly APIs — creativity and engineering skill matter more.

1

Idea Discussion Meeting

By September 20 — Meet with the instructor to discuss and finalize your project idea. Come prepared with a concept and initial scope.

2

Mid Presentation

Present your project's progress **after the mid term exam**. Demonstrate working features, architecture decisions, and team.

3

Final Presentation

Full project demo with report and viva **before the final exam**. This is your chance to showcase everything you've built and learned.



How to Succeed in This Course

Success in CS325 comes down to consistent habits. These three principles will carry you through the semester.



Attentive Listening

Lectures are your primary resource. Concepts build on each other — grab them the first time. Ask questions when something is unclear.



Attend All Classes

Consistency is crucial. Missing even one lecture can create gaps that compound. Attendance is mandatory under university policy.



Be On Time

Arriving on time shows respect for the class and keeps you in the right mindset. Late arrivals disrupt your focus and others'.

What Is Software Engineering?

Before we dive in, let's ground ourselves in the fundamentals. Software engineering is more than writing code — it's a disciplined approach to building systems that solve real problems.

Engineering

The systematic application of science and mathematics to design and improve systems, devices, or processes that solve problems under constraints. *(Wikipedia)*

Software

A set of instructions and documentation that tells a computer what to do. Includes **application software** (Facebook, Word, delivery apps) and **system software** (firmware, OS).

Software Engineering

An engineering discipline concerned with **all aspects of software production** — from initial conception through operation and maintenance.

- 📄 Software is not just programs — it also includes all documentation needed by users, QA staff, and developers.

Myths of Software Engineering

Let's debunk common misconceptions — especially those fueled by the AI hype cycle.

✗ "SE is only coding."

SE spans requirements, design, testing, documentation, and maintenance — coding is just one piece.

✗ "We don't need SE because of AI."

As codebases grow, complexity increases. SE fundamentals organize that complexity. AI models depend on quality input — complex prompts increase hallucinations.

✗ "SE is just prompt engineering."

We must steer AI agents with long-term thinking. Prompt engineering is a tool, not a discipline.

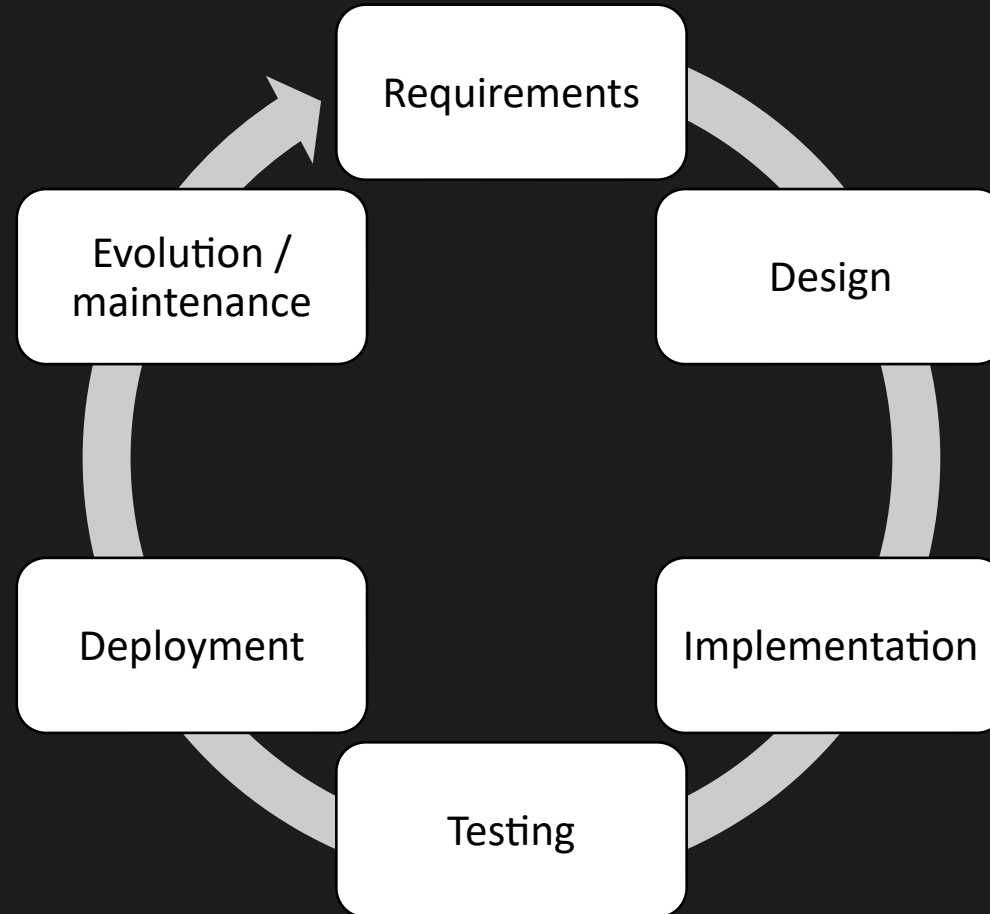
✗ "AI agents replace software engineers."

AI agents have short-term memory and limited context windows. Human judgment is irreplaceable.

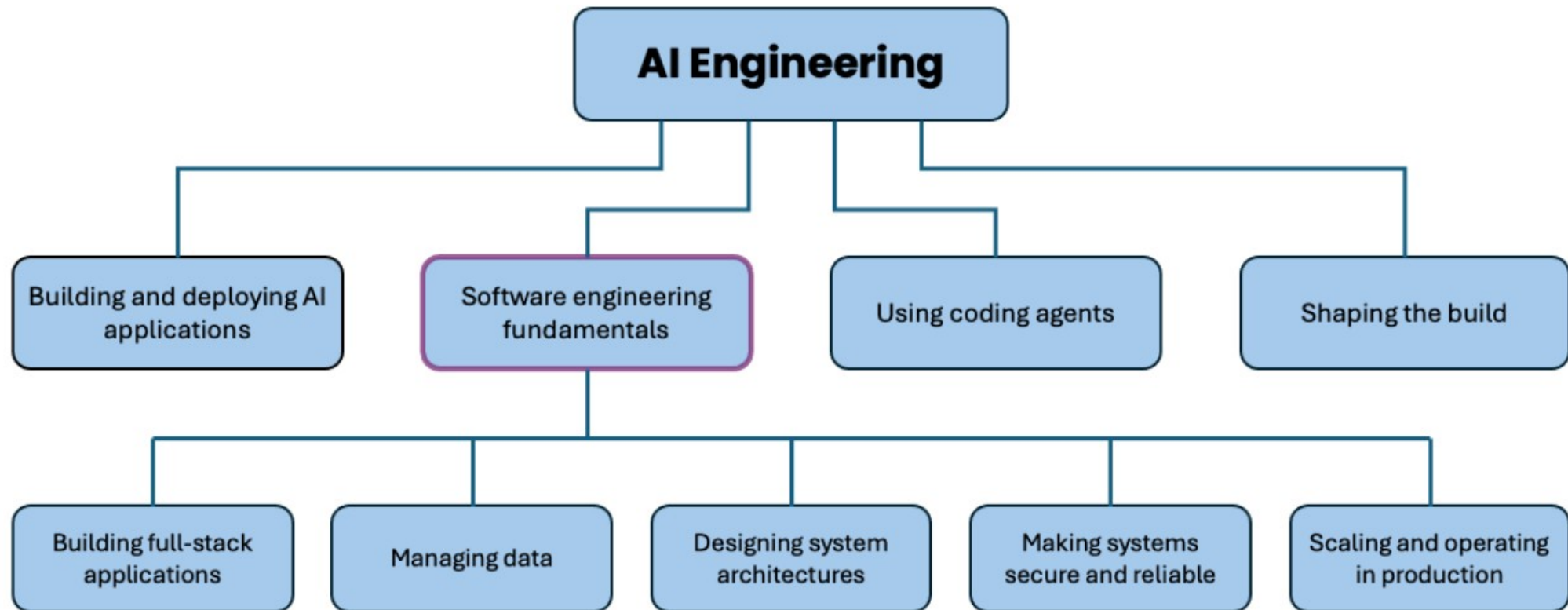
✗ "SE is only for software engineers."

AI engineers, data engineers, and any technical role building systems need SE principles.

Software Development LifeCycle SDLC



AI Engineering Skills Map: Software Engineering Fundamentals



Summary

- Software engineering is an engineering discipline that is concerned with all aspects of **software production**.
- **Software is not just a program** or programs but also includes all electronic documentation that is needed by system users, quality assurance staff, and developers. Essential software product attributes are maintainability, dependability and security, efficiency, and acceptability.
- The **software development lifecycle (SDLC)** is a structured and iterative methodology used by development teams to build, deliver and maintain high-quality and cost-effective software systems.
- The **software process** includes all of the activities involved in software development. The high-level activities of specification, development, validation, and evolution are part of all software processes.